

Vision-Based Pothole Detection and Severity Classification

Combining YOLOv8 Segmentation, Monocular Depth Estimation,
Rule-Based and ML-Based Severity Classification

CVCSL7360 — Computer Vision Course Project

April 5, 2026

Abstract

This report documents a vision-based pipeline for detecting potholes and estimating their severity from monocular road images. The system combines YOLOv8 instance segmentation for pothole localisation, Depth-Anything-V2 for monocular depth estimation, and two complementary severity classification approaches: a rule-based manual threshold classifier and a suite of lightweight ML classifiers (Logistic Regression, Random Forest, SVM, Naive Bayes). Since no ground-truth severity labels exist, pseudo-labels are generated through KMeans clustering on extracted features. We evaluate all ML models via classification reports and bootstrapped confidence intervals.

Contents

1	Introduction	3
2	Dataset	3
3	Pipeline Architecture	3
3.1	Stage 1: Pothole Segmentation	4
3.2	Stage 2: Depth Estimation	4
3.3	Stage 3: Feature Extraction	4
3.4	Stage 4: Severity Classification	4
3.4.1	4a: Rule-Based Classifier (Manual Thresholds)	4
3.4.2	4b: ML-Based Classifiers	5
4	Pseudo-Label Generation via KMeans	5
4.1	Procedure	5
4.2	Label Distribution	6
4.3	Visualisation	6
4.4	Limitations	6
5	Training	7
5.1	Setup	7
5.2	Training Procedure	7

6	Evaluation	7
6.1	Validation Accuracy	7
6.2	Bootstrapped Confidence Intervals	8
6.3	Discussion	8
7	Inference Pipeline	8
7.1	Workflow	8
7.2	Sample Inference Outputs	9
8	File Structure	10
9	Conclusion	10

1 Introduction

Road surface deterioration, particularly potholes, poses significant safety hazards. Automated detection and severity assessment can assist municipal road maintenance programs in prioritising repairs. This project develops an end-to-end pipeline that:

1. **Detects** potholes via YOLOv8 instance segmentation.
2. **Estimates depth** using a pretrained monocular depth model (Depth-Anything-V2).
3. **Extracts features** combining geometric (area, bounding box) and depth statistics.
4. **Classifies severity** via (a) rule-based thresholds and (b) trained ML models.

2 Dataset

The primary dataset is sourced from Kaggle¹ and exported via Roboflow in YOLOv8 segmentation format.

Table 1: Dataset split sizes

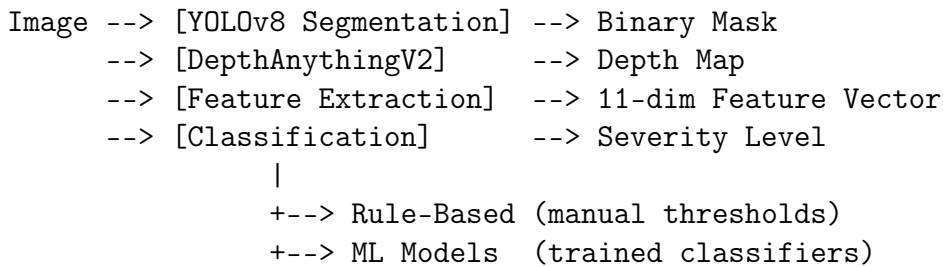
Split	Images	Labels
data1/train	720	720
data1/valid	60	60

Each label file follows YOLO segmentation format: one line per object with a class index followed by normalised polygon coordinates.

Depth maps are precomputed for all images using Depth-Anything-V2 and stored as .npy files in the `depth_maps_1/` directory. Since a single image may contain multiple potholes, each pothole is treated as a separate sample, yielding 1846 training samples and 201 validation samples.

3 Pipeline Architecture

The full system is composed of four stages, illustrated below:



¹<https://www.kaggle.com/datasets/farzadnekouei/pothole-image-segmentation-dataset>

3.1 Stage 1: Pothole Segmentation

We use a fine-tuned YOLOv8 instance segmentation model (`yolo-segmentation/model/best.pt`) trained on the pothole dataset. Given an input image, the model outputs binary segmentation masks. When multiple pothole instances are detected, the mask with the largest foreground area is selected for single-image analysis; for ML training, all detected instances are processed individually.

3.2 Stage 2: Depth Estimation

Monocular depth is estimated using Depth-Anything-V2 (ViT-S encoder). The model produces a relative depth map for each image. For batch processing (ML training), depth maps are precomputed and normalised to $[0, 1]$ per image:

$$d_{\text{norm}} = \frac{d - d_{\min}}{d_{\max} - d_{\min} + \epsilon} \quad (1)$$

where $\epsilon = 10^{-8}$ prevents division by zero.

Important caveat: Since normalisation is per-image, the maximum depth value for any image containing a pothole as its deepest point will always be 1.0. This reduces the discriminative power of depth features across images.

3.3 Stage 3: Feature Extraction

For each detected pothole (mask region), 11 features are extracted:

Table 2: Extracted feature set

Category	Feature	Description
5*Geometric	height	Bounding box height (pixels)
	width	Bounding box width (pixels)
	box_area	height $\times$ width
	pothole_area	Pixel count where mask = 1
	nonpothole_area	box_area $-$ pothole_area
6*Depth	mean_depth	Mean depth within mask
	max_depth	Maximum depth within mask
	min_depth	Minimum depth within mask
	depth_std	Standard deviation of depth
	depth_range	max_depth $-$ min_depth
	p90_depth	90th percentile depth

3.4 Stage 4: Severity Classification

Two classification approaches run in parallel:

3.4.1 4a: Rule-Based Classifier (Manual Thresholds)

The original classifier uses hand-crafted rules operating on `max_depth` and `area`:

```

if area <= 0:
    return "No_pothole"
if max_depth < 0.3 and area < 500:
    return "Shallow"
if max_depth < 0.6:
    return "Moderate"
return "Deep"

```

3.4.2 4b: ML-Based Classifiers

Four scikit-learn classifiers are trained on the 11-dimensional feature vector:

1. **Logistic Regression** (max_iter=1000)
2. **Random Forest** (100 estimators)
3. **SVM** (RBF kernel, probability enabled)
4. **Gaussian Naive Bayes**

All features are standardised using **StandardScaler** (fit on training data, applied to validation data). Trained models and the scaler are persisted as **.pkl** files in **ml_models/**.

4 Pseudo-Label Generation via KMeans

Since the dataset lacks ground-truth severity annotations, we generate pseudo-labels using unsupervised clustering.

4.1 Procedure

1. Select two features indicative of severity: **max_depth** and **pothole_area**.
2. Apply **StandardScaler** to normalise these features (zero mean, unit variance).
3. Fit KMeans with $k = 3$ clusters on the training set.
4. Order clusters by severity: compute the L2 norm of each cluster centroid in the scaled feature space. The cluster with the smallest norm is labeled *Shallow* (Level 0), the middle one *Moderate* (Level 1), and the largest *Deep* (Level 2).
5. For the validation set: transform features using the same scaler, then assign each sample to its nearest cluster centroid from the training-fitted KMeans model.

4.2 Label Distribution

Table 3: Pseudo-label distribution across splits

Severity	Train	Valid	% (Train)
Shallow	733	65	39.7%
Moderate	922	118	49.9%
Deep	191	18	10.3%
Total	1846	201	100%

The distribution is imbalanced, with the *Deep* class comprising only $\sim 10\%$ of training samples.

4.3 Visualisation

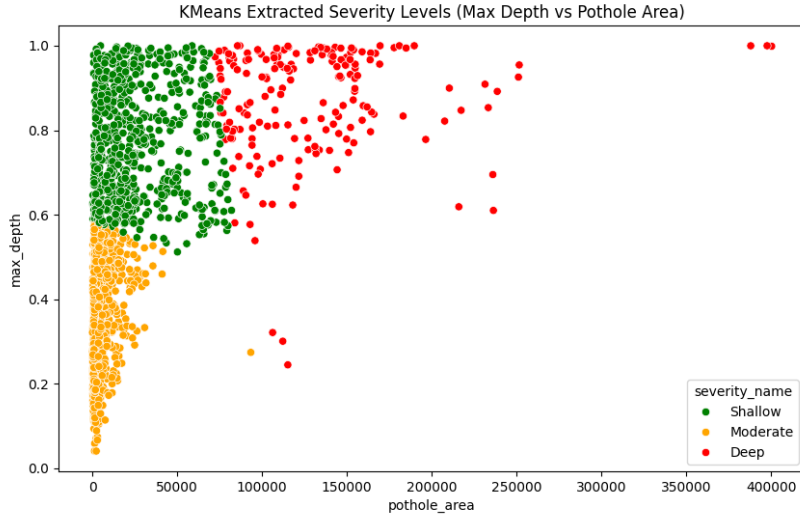


Figure 1: KMeans cluster assignments visualised as Max Depth vs. Pothole Area. Colours indicate the assigned severity level.

4.4 Limitations

- **Circular evaluation:** Since pseudo-labels are derived from the same feature space used for training, high classifier accuracy is expected but does not indicate generalisation to true severity.
- **Depth normalisation artifact:** Per-image normalisation causes $\text{max_depth} \approx 1.0$ for most potholes, making clustering effectively area-dominant.
- **No absolute depth:** Monocular depth estimation provides relative, not metric, depth values.

5 Training

5.1 Setup

- **Training samples:** 1846 (from 720 images, multiple potholes per image)
- **Validation samples:** 201 (from 60 images)
- **Feature scaling:** `StandardScaler` fit on training data
- **Random seed:** 42 for reproducibility

5.2 Training Procedure

Each classifier is trained on the 11-dimensional standardised feature vector. The training script (`ml_classifier.py`) performs the following:

1. Extract features from all `data1/train` images using YOLO labels and precomputed depth maps.
2. Generate pseudo-labels via KMeans (Section 4).
3. Fit each of the four classifiers on the training set.
4. Evaluate on the validation set.
5. Save all models and the scaler to `ml_models/`.
6. Save extracted features to CSV files in `ml_results/`.

6 Evaluation

6.1 Validation Accuracy

All four models are evaluated on the 201-sample validation set using accuracy and macro F1-score.

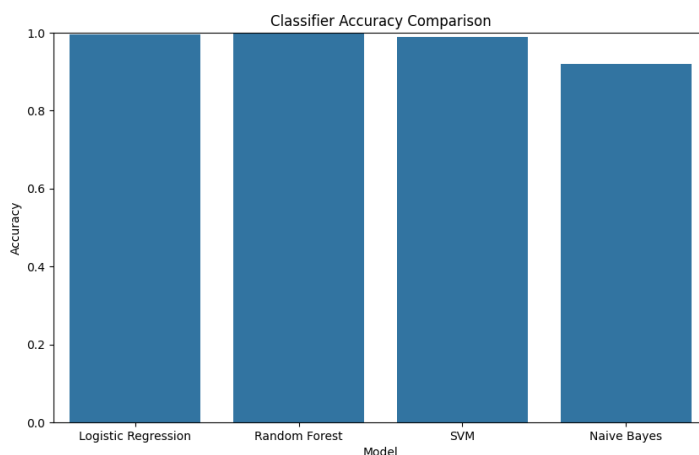


Figure 2: Validation accuracy comparison across the four ML classifiers.

6.2 Bootstrapped Confidence Intervals

To assess robustness, we perform bootstrap resampling ($n = 1000$ iterations) on the validation set. For each iteration, a bootstrap sample (with replacement) of the same size is drawn, and accuracy is computed.

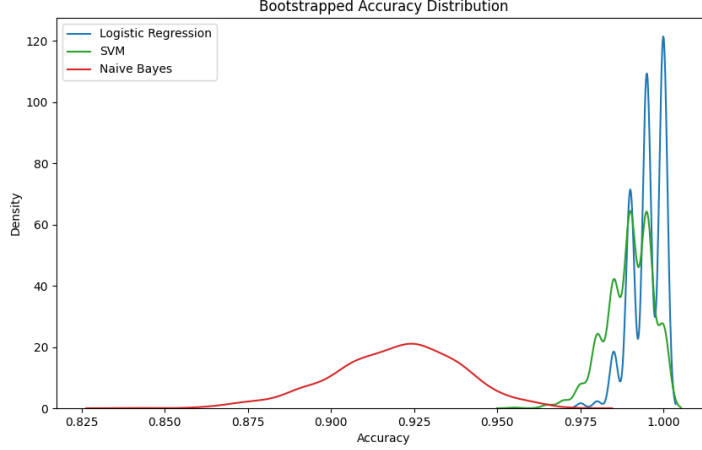


Figure 3: Bootstrapped accuracy distributions for all four classifiers. Narrow, overlapping distributions indicate similar and stable performance.

6.3 Discussion

The high validation accuracy ($> 95\%$) across all models is expected given the circular nature of pseudo-label evaluation (Section 4). The ML models effectively learn to replicate the KMeans decision boundaries, which are themselves a simple function of the input features. This is a known limitation when no ground-truth severity labels are available.

Despite this, the framework demonstrates a valid methodology: if real severity annotations were available, the same pipeline could be directly retrained with meaningful supervision.

7 Inference Pipeline

The unified inference script (`inference.py`) processes a single image through the full pipeline and produces predictions from both the rule-based and ML-based classifiers.

7.1 Workflow

1. **Segmentation:** YOLOv8 produces the pothole binary mask.
2. **Depth estimation:** Depth-Anything-V2 generates the depth map on-the-fly.
3. **Feature extraction:** Both the rule-based features (`max_depth`, `area`) and the 11-dim ML feature vector are computed.
4. **Classification:**
 - Rule-based classifier applies manual thresholds.

- All four saved ML models are loaded, features are scaled, and predictions are made.

5. **Visualisation:** A 3-panel figure is generated:

- Left: Original image with severity labels overlaid.
- Centre: Pothole mask overlay (semi-transparent red highlight with bounding box).
- Right: Depth map heatmap.

7.2 Sample Inference Outputs

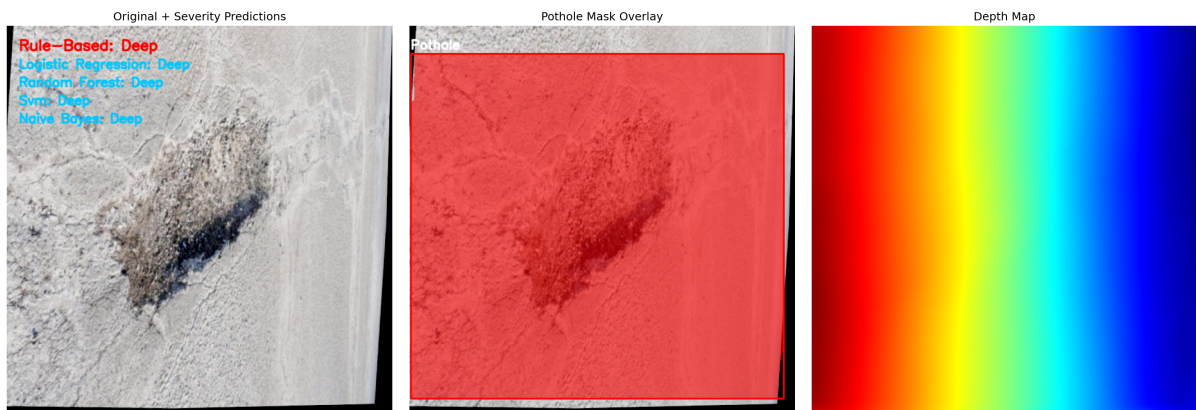


Figure 4: Inference output for a “Deep” severity pothole. Left: original with predictions, Centre: mask overlay, Right: depth heatmap.

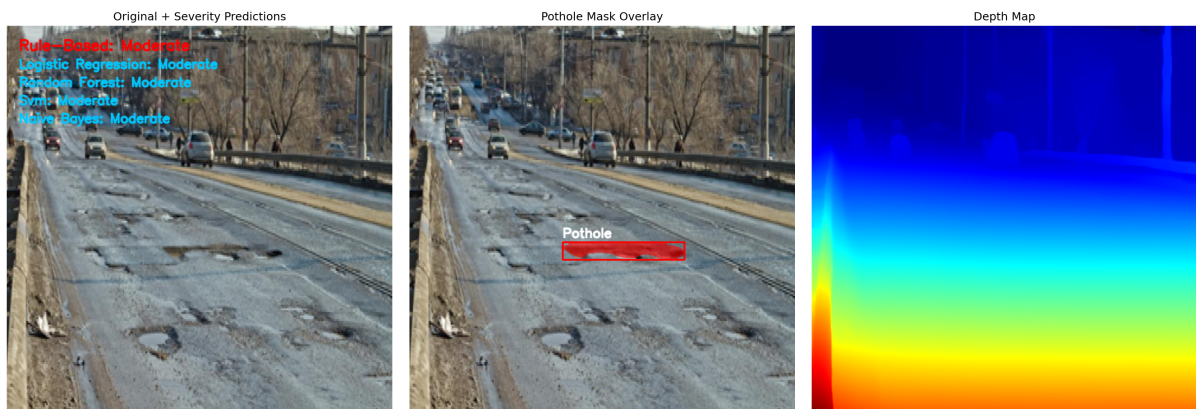


Figure 5: Inference output for a “Moderate” severity pothole.

8 File Structure

Table 4: Project file descriptions

File	Purpose
<code>main.py</code>	Original single-image pipeline (rule-based only)
<code>inference.py</code>	Unified pipeline (rule-based + ML classifiers)
<code>segmentation.py</code>	YOLOv8 segmentation mask extraction
<code>features.py</code>	Depth feature extraction from mask + depth map
<code>classifier.py</code>	Rule-based severity thresholds
<code>ml_classifier.py</code>	ML training: feature extraction, KMeans, model training
<code>depth/generate_depth.py</code>	Batch depth map generation (saves <code>.npy</code>)
<code>ml_models/*.pkl</code>	Saved trained models and feature scaler
<code>ml_results/*.png</code>	Evaluation plots (accuracy, bootstrap, KMeans)
<code>ml_results/*.csv</code>	Extracted feature tables

9 Conclusion

We presented a modular pipeline for pothole detection and severity estimation that combines deep learning based perception (YOLOv8 segmentation: Depth-Anything-V2 depth estimation) with lightweight classical ML classifiers. The key contributions are:

1. A complete pipeline from raw image to severity label, integrating segmentation, depth, and classification.
2. An unsupervised pseudo-labelling approach using KMeans clustering when ground-truth severity annotations are unavailable.
3. A comparative study of four ML classifiers with bootstrapped evaluation.
4. A unified inference script that runs both rule-based and ML-based classification in parallel for comparison.

Future Work: Obtaining real severity annotations would enable truly supervised training and meaningful evaluation. Additionally, using metric depth estimation (e.g., from stereo cameras or LiDAR) would provide absolute depth values, significantly improving depth-based severity discrimination.